

CryptoV4ult Enterprise Security Review



Mohamad Abdelnaby
6/29/2024



Section One: Integrating SDLC



Transitioning to Secure SDLC

Requirements Analysis

- Conduct user interviews to gather functional requirements.
- Write a requirements document for task management features.
- Risk assessment

Design

- Create a high-level architecture diagram for the application.
- Design the database schema for tasks.
- Threat Modeling and Design Review



Transitioning to Secure SDLC

Development

- Code the user interface using HTML and CSS.
- Implement interactive elements using JavaScript.
- Set up a Flask application to handle API requests.
- Implement CRUD operations for tasks.
- Static Analysis

Testing

- Write and execute functional test cases.
- Conduct browser compatibility testing.
- Security Testing and Code Review



Transitioning to Secure SDLC

Deployment

- Deploy the application to Heroku.
- Perform smoke testing on the deployed application.
- Security Assessment and Secure Configurations

Maintenance

- Monitor application logs and fix reported issues.
- Gather user feedback for future feature additions.
- Operational Assurance



Advocating for Secure SDLC

1. Discover vulnerabilities early on

*With the waterfall model typically you would ship the project as a whole at the deadline, however with the Secure SDLC you're shipping more often, and by implementing the **Secure** part you're discovering vulnerabilities and fixing them more often*

2. Reduce costs

By fixing vulnerabilities early on when they are not yet exploitable you are saving yourself a lot of cost, money, and effort on the long run.

3. Prevent Legal fines and actions

By fixing vulnerabilities that are discovered early on, you are saving yourself from the burden of being open to legal fines or lawsuits in case of compromise

4. Policies compliant

You are being compliant with legal policies if you ship software that isn't vulnerable as you are not putting the customers' data at risk.

5. Continuously monitoring for vulnerabilities & Testing

*You will discover vulnerabilities and patch them far more often.
Big cost on the short term, significantly low cost on the long term.*



Section Two:

Vulnerabilities and Remediation



Vulnerabilities and remediation

1. SQL Injections
Description
<i>Basically the SQL injection works when:</i> • <i>Dynamic queries or non-parameterized calls without context-aware escaping are used directly in the interpreter.</i> • <i>Hostile data is directly used or concatenated. The SQL or command contains the structure and malicious data in dynamic queries, commands, or stored procedures.</i>
Risk
<i>Attacker can escape the login entirely and gain access directly to the DB, and can even take print out all the data from the DB.</i>
Remediation
• <i>Use positive server-side input validation. This is not a complete defense as many applications require special characters, such as text areas or APIs for mobile applications.</i> • <i>Use LIMIT and other SQL controls within queries to prevent mass disclosure of records in case of SQL injection.</i>



Vulnerabilities and remediation

2. Broken Authentication

Description

Access control enforces policy such that users cannot act outside of their intended permissions. Failures typically lead to unauthorized information disclosure, modification, or destruction of all data or performing a business function outside the user's limits.

Risk

- *Violation of the principle of least privilege or deny by default, where access should only be granted for particular capabilities, roles, or users, but is available to anyone.*
- *Bypassing access control checks by modifying the URL (parameter tampering or force browsing), internal application state, or the HTML page, or by using an attack tool modifying API requests.*
- *Permitting viewing or editing someone else's account, by providing its unique identifier (insecure direct object references)*
- *Accessing API with missing access controls for POST, PUT and DELETE.*
- *Elevation of privilege. Acting as a user without being logged in or acting as an admin when logged in as a user.*

Remediation

- *Except for public resources, deny by default.*
- *Implement access control mechanisms once and re-use them throughout the application, including minimizing Cross-Origin Resource Sharing (CORS) usage.*
- *Model access controls should enforce record ownership rather than accepting that the user can create, read, update, or delete any record.*
- *Log access control failures, alert admins when appropriate (e.g., repeated failures).*
- *Rate limit API and controller access to minimize the harm from automated attack tooling.*

3. Identification and Authentication Failures

Description

Confirmation of the user's identity, authentication, and session management is critical to protect against authentication-related attacks.

Risk

- *Permits automated attacks such as credential stuffing, where the attacker has a list of valid usernames and passwords.*
- *Permits brute force or other automated attacks.*
- *Permits default, weak, or well-known passwords, such as "Password1" or "admin/admin".*
- *Uses weak or ineffective credential recovery and forgot-password processes, such as "knowledge-based answers," which cannot be made safe.*
- *Uses plain text, encrypted, or weakly hashed passwords data stores*
- *Has missing or ineffective multi-factor authentication.*
- *Exposes session identifier in the URL.*

Remediation

- *Where possible, implement multi-factor authentication to prevent automated credential stuffing, brute force, and stolen credential reuse attacks.*
- *Do not ship or deploy with any default credentials, particularly for admin users.*
- *Implement weak password checks, such as testing new or changed passwords against the top 10,000 worst passwords list.*
- *Align password length, complexity, and rotation policies with National Institute of Standards and Technology (NIST) 800-63b's guidelines in section 5.1.1 for Memorized Secrets or other modern, evidence-based password policies.*
- *Ensure registration, credential recovery, and API pathways are hardened against account enumeration attacks by using the same messages for all outcomes.*
- *Limit or increasingly delay failed login attempts, but be careful not to create a denial of service scenario. Log all failures and alert administrators when credential stuffing, brute force, or other attacks are detected.*
- *Use a server-side, secure, built-in session manager that generates a new random session ID with high entropy after login. Session identifier should not be in the URL, be securely stored, and invalidated after logout, idle, and absolute timeouts.*



Threat Matrix

Pathway (Vulnerability)	Impact Level	Likelihood Level
SQL Injections	High	High
Broken Authentication	Medium	Medium
Identification and Authentication Failures	Low	Low

Impact	Low	Medium	High
Likelihood			
High			SQL Injections
Medium		Broken Authentication	
Low	Identification and Authentication Failures		



Section Three: Container Security



Trivy scan screenshot

Place a screenshot from the Trivy scan results on this slide.

coreutils	CVE-2014-9471	HIGH	8.13-3.5		coreutils: memory corruption flaw in parse_datetime() →avd.aquasec.com/nvd/cve-2014-9471
	CVE-2016-2781	MEDIUM			coreutils: Non-privileged session can escape to the parent session in chroot →avd.aquasec.com/nvd/cve-2016-2781
gcc-4.7-base	CVE-2014-5044	CRITICAL	4.7.2-5		gcc: integer overflow flaws in libgfortran →avd.aquasec.com/nvd/cve-2014-5044
	CVE-2002-2439	HIGH			gcc: Integer overflow can occur during the computation of the memory region... →avd.aquasec.com/nvd/cve-2002-2439
	CVE-2017-11671	MEDIUM			gcc: GCC generates incorrect code for RDRAND/RDSEED intrinsics →avd.aquasec.com/nvd/cve-2017-11671
gnupg	CVE-2015-1607		1.4.12-7+deb7u9		gnupg2: memcpy with overlapping ranges (keybox_search.c) →avd.aquasec.com/nvd/cve-2015-1607
gpgv					
inetutils-ping	CVE-2014-3634	HIGH	2:1.9-2		rsyslog: remote syslog PRI vulnerability →avd.aquasec.com/nvd/cve-2014-3634
libapr1	CVE-2017-12613		1.4.6-3+deb7u1	1.4.6-3+deb7u2	apr: Out-of-bounds array deref in apr_time_exp*() functions →avd.aquasec.com/nvd/cve-2017-12613
libaprutil1	CVE-2017-12618	MEDIUM	1.4.1-3	1.4.1-3+deb7u1	apr-util: Out-of-bounds access in corrupted SDBM database →avd.aquasec.com/nvd/cve-2017-12618
libaprutil1-dbd-sqlite3					
libaprutil1-ldap					
libblkid1	CVE-2014-9114	HIGH	2.20.1-5.3		util-linux: command injection flaw in blkid →avd.aquasec.com/nvd/cve-2014-9114
	CVE-2016-5011	MEDIUM			util-linux: Extended partition loop in MBR partition table leads to DOS →avd.aquasec.com/nvd/cve-2016-5011



Report to Fix Container Issues

Fill out the report with at least 7 items. Make sure to write the **Issues in the correct form of (Application Name: CVE number)**.

Issues	Unpatched Software Version	Patched Software Version
Apache2: CVE-2018-1312	2.2.22-13+deb7u12	2.2.22-13+deb7u13
Bash: CVE-2014-6271	4.2+dfsg-0.1	4.2+dfsg-0.1+deb7u1
Libssl1.0.0: CVE-2017-3735	1.0.1t-1+deb7u2	1.0.1t-1+deb7u3
Openssl: CVE-2017-3735	1.0.1t-1+deb7u2	1.0.1t-1+deb7u3
Libprocps0: CVE-2018-1126	1:3.3.3-3	1:3.3.3-3+deb7u1
Perl: CVE-2018-6913	5.14.2-21+deb7u5	5.14.2-21+deb7u6
Libapr1: CVE-2017-12613	1.4.6-3+deb7u1	1.4.6-3+deb7u2



Section Four: API Security

API Vulnerabilities and remediation



1. Broken Authentication

Description

- *Permits credential stuffing where the attacker uses brute force with a list of valid usernames and passwords.*
- *Permits attackers to perform a brute force attack on the same user account, without presenting captcha/account lockout mechanism.*
- *Permits weak passwords.*
- *Sends sensitive authentication details, such as auth tokens and passwords in the URL.*
- *Allows users to change their email address, current password, or do any other sensitive operations without asking for password confirmation.*
- *Doesn't validate the authenticity of tokens.*

Risk

Attackers can gain complete control of other users' accounts in the system, read their personal data, and perform sensitive actions on their behalf. Systems are unlikely to be able to distinguish attackers' actions from legitimate user ones.

Remediation

- *Make sure you know all the possible flows to authenticate to the API (mobile/ web/deep links that implement one-click authentication/etc.). Ask your engineers what flows you missed.*
- *Read about your authentication mechanisms. Make sure you understand what and how they are used. OAuth is not authentication, and neither are API keys.*
- *Don't reinvent the wheel in authentication, token generation, or password storage. Use the standards.*
- *Credential recovery/forgot password endpoints should be treated as login endpoints in terms of brute force, rate limiting, and lockout protections.*
- *Require re-authentication for sensitive operations (e.g. changing the account owner email address/2FA phone number).*
- *Where possible, implement multi-factor authentication.*



API Vulnerabilities and remediation

2. Unrestricted Resource Consumption

Description

Satisfying API requests requires resources such as network bandwidth, CPU, memory, and storage. Sometimes required resources are made available by service providers via API integrations, and paid for per request, such as sending emails/SMS/phone calls, biometrics validation, etc.

An API is vulnerable if at least one of the following limits is missing or set inappropriately (e.g. too low/high):

- *Execution timeouts*
- *Maximum allocable memory*
- *Maximum number of file descriptors*
- *Maximum number of processes*

Risk

Exploitation can lead to DoS due to resource starvation, but it can also lead to operational costs increase such as those related to the infrastructure due to higher CPU demand, increasing cloud storage needs, etc.

Remediation

- *Use a solution that makes it easy to limit memory, CPU, number of restarts, file descriptors, and processes such as Containers / Serverless code (e.g. Lambdas).*
- *Define and enforce a maximum size of data on all incoming parameters and payloads, such as maximum length for strings, maximum number of elements in arrays, and maximum upload file size (regardless of whether it is stored locally or in cloud storage).*
- *Implement a limit on how often a client can interact with the API within a defined timeframe (rate limiting).*
- *Rate limiting should be fine tuned based on the business needs. Some API Endpoints might require stricter policies.*
- *Limit/throttle how many times or how often a single API client/user can execute a single operation (e.g. validate an OTP, or request password recovery without visiting the one-time URL).*



API Vulnerabilities and remediation

3. [Name of the vulnerability]

Description

Attackers will often attempt to find unpatched flaws, common endpoints, services running with insecure default configurations, or unprotected files and directories to gain unauthorized access or knowledge of the system. Most of this is public knowledge and exploits may be available.

- *Appropriate security hardening is missing across any part of the API stack, or if there are improperly configured permissions on cloud services*
- *The latest security patches are missing, or the systems are out of date*
- *Unnecessary features are enabled (e.g. HTTP verbs, logging features)*
- *There are discrepancies in the way incoming requests are processed by servers in the HTTP server chain*
- *Transport Layer Security (TLS) is missing*

Risk

Security misconfigurations not only expose sensitive user data, but also system details that can lead to full server compromise.

Remediation

- *The API life cycle should include:*
- *A repeatable hardening process leading to fast and easy deployment of a properly locked down environment*
- *A task to review and update configurations across the entire API stack. The review should include: orchestration files, API components, and cloud services (e.g. S3 bucket permissions)*
- *An automated process to continuously assess the effectiveness of the configuration and settings in all environments*
- *Ensure that all API communications from the client to the API server and any downstream/upstream components happen over an encrypted communication channel (TLS), regardless of whether it is an internal or public-facing API.*
- *Be specific about which HTTP verbs each API can be accessed by: all other HTTP verbs should be disabled (e.g. HEAD).*